

We'll use a few of the available customizations to improve this plot's readability. Let's start by adding a title and labeling the axes:

```
mpl_squares.py import matplotlib.pyplot as plt

squares = [1, 4, 9, 16, 25]

fig, ax = plt.subplots()
❶ ax.plot(squares, linewidth=3)

# Set chart title and label axes.
❷ ax.set_title("Square Numbers", fontsize=24)
❸ ax.set_xlabel("Value", fontsize=14)
   ax.set_ylabel("Square of Value", fontsize=14)

# Set size of tick labels.
❹ ax.tick_params(labelsize=14)

plt.show()
```

The `linewidth` parameter controls the thickness of the line that `plot()` generates ❶. Once a plot has been generated, there are many methods available to modify the plot before it's presented. The `set_title()` method sets an overall title for the chart ❷. The `fontsize` parameters, which appear repeatedly throughout the code, control the size of the text in various elements on the chart.

The `set_xlabel()` and `set_ylabel()` methods allow you to set a title for each of the axes ❸, and the method `tick_params()` styles the tick marks ❹. Here `tick_params()` sets the font size of the tick mark labels to 14 on both axes.

As you can see in Figure 15-2, the resulting chart is much easier to read. The label type is bigger, and the line graph is thicker. It's often worth experimenting with these values to see what works best in the resulting graph.

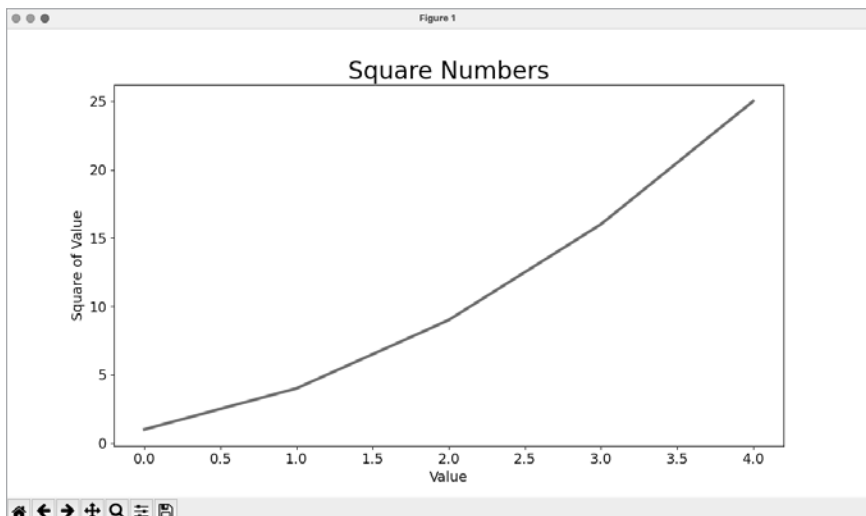


Figure 15-2: The chart is much easier to read now.